iPRA with inferred witnesses

Three inference families, with equality supplied by a fixed numerical probe

Research note prepared for Vladimir Reshetnikov

11 November 2026

Abstract

We give a further reduction of the current ConciseIPRA presentation. Its new elimination rule derives a numerical name from the zero and unit instances of an existential matrix and specializes the opened clause at that name. The certificate supplies neither a replacement term nor a proof of the naming equality. A two-row numerical package recovers arbitrary equality transport, while a separate substitution argument recovers ordinary existential elimination even when substitution creates the naming pattern. There is one fixed reflexivity clause and three inference families: clause use, paired introduction, and opening with an inferred witness. Terms are fully normalized scalar iterators; formulas are equations and a single paired binder. We prove equality of the entire canonical clause consequence relation with the established presentation. Composing with its arithmetic bridge gives bi-interpretability with full first-order intuitionistic primitive recursive arithmetic, with whole numerical domains, actual equality, and identity numerical composites. A standalone Python checker, a fuel-bounded variant, concrete certificates, and reproducible measurements accompany the proof. The selected checker has 1,990 normalized bytes. The size improvement and the mathematical inference restriction are reported separately. No absolute minimality theorem or new proof-assistant verification is claimed.

Contents

1 Result and the comparison that it supports 3

2 Synthesizing a witness name when its clause is opened 4

2.1 Complete presentation 4

2.2 The preliminary rules that do not need general substitution 5

2.3 The substitution theorem and its noncircular proof 6

2.4 Recovering every equality rule 7

2.5 Identity-language equivalence and both interpretation composites 8

3 The normalization layer is part of the theory 9

4 The full intuitionistic theory and both composites 11

4.1 Ordinary realization 11

4.2 All logical operations as macros 11

4.3 The numerical arithmetic bridge 12

4.4 Why this is bi-interpretability 13

5 Complete implementation and measured simplification 13

5.1 Input contract and trusted code 13

5.2 Source measurements 14

5.3 The fuel-bounded certificate relation 15

5.4 Executed evidence 15

A Selected complete checker 16

B Reproduction and source attribution 17

1 Result and the comparison that it supports

The construction retains the numerical sort, actual equality, and ordinary reading of the paired binder from ConciseIPRA. Its change is local but mathematical: an equality proof that previously had to be supplied at witness discharge is replaced by information already contained in the proved major formula. This is an admissible restriction of the equality interface, with a proof of completeness for arbitrary finite contexts. It does not rely on agreement on standard natural numbers alone.

The fixed baseline. All repository comparisons refer to the following commit of `VladimirReshetnikov/ProveIt`:

`17898ea7e6a1300dfdcad71040d5daeed67916d4`

The relevant directory is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA` [1]. The README at that commit describes 67 filed packages. We also inspected the six ZIP packages in its `docs/incoming` directory. In particular, the incoming `witness_discharge` proposal already has three inference families and an explicit reflexivity clause [2]; claiming that rule count itself as new would be incorrect. Its elimination requires a major proof, an equality proof, a branch proof, and an explicit replacement term. The new rule needs only a major proof and a branch proof.

What is new. The fixed zero/unit test for an inferred witness, its use to specialize the clause before opening, and its complete substitution and equality recovery proofs are the new mathematical content. In particular, a syntactic test need not be reflected by arbitrary substitution: preservation of its positive case, together with strong induction on branch proofs, suffices. Full normalization, optional-antecedent clauses, the reflexivity clause, the single witness-selected introduction, numerical selector packages, and the scalar arithmetic bridge are inherited and attributed.

What "simpler" means here. The proposal removes a proof premise and a supplied numerical argument from the strongest inspected equality-discharge interface, and the delivered complete implementation is smaller than the inspected explicitly typed incoming checkers. The proof representation also changes from checking many repeated conclusions to inferring each conclusion once. A matched control quantifies the cost of the new naming test separately from that representation change (Section 5). The repository's much shorter compact codecs remain shorter. We do not call this an absolute source-size minimum, and do not infer a lower bound from the absence of a smaller draft.

Strength and verification boundary. The intended iPRA is the full intuitionistic first-order theory with a symbol for each finite primitive-recursive description, its defining equations, the usual successor axioms, and induction for quantifier-free formulas with parameters and retained assumptions. Induction is not added for arbitrary quantified formulas. The new equivalence is a written universal proof. The accompanying executions check the implementation and specific certificates. Existing Lean sources certify the inherited bridge; this package does not claim a new Lean proof of inferred opening.

2 Synthesizing a witness name when its clause is opened

The new rule substitutes its inferred numerical name into the opened clause, so its branch proves the desired outer conclusion directly. The rule requires neither an equality proof, a supplied replacement term, nor a separate branch-result formula. The main completeness issue is admissibility of substitution. We resolve it before recovering unrestricted existential-style elimination, using only two guard-free forms of elimination that are immediately available.

2.1 Complete presentation

Numerical terms are full computation normal forms of

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h)$$

under

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u].$$

Only u in h is bound. Write N for normalization. Substitution is simultaneous, capture avoiding, and followed by N. Inserting a term does not recursively substitute into that inserted copy. The inherited termination, confluence, and substitution-coherence results for N are part of the metatheoretic foundation and of the trusted checker boundary.

Formulas and optional-premise clauses are

$$A ::= (s = t) \mid D_{x,y}B, \qquad [\Delta; y.B(y)], \quad |\Delta| \in \{0, 1\}, \quad y \notin \mathrm{FV}(\Delta).$$

Each context is a finite list of clause occurrences containing the fixed axiom occurrence $\mathcal{R} = [\,; y.y = y]$. An ordinary assumption A means the unconditional constant clause $[\,; v.A]$. Put $H = (0 = 0)$, $1 = S0$, and $2 = S(S0)$.

The intended ordinary readings are actual numerical equality,

$$(D_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)),$$

and $[\,; y.B]^\circ = \forall y B^\circ(y)$, $[[A]; y.B]^\circ = A^\circ \to \forall y B^\circ(y)$. The three primitive inference families are the following.

Use. For a retained clause $[\Delta; y.B] \in \Gamma$,

$$\frac{\{\,\Gamma \vdash A : A \text{ is the antecedent occurrence in } \Delta\,\}}{\Gamma \vdash N(B(t))}. \tag{U}$$

An empty antecedent requires no proof; a singleton requires one. In particular, a use of $\mathcal{R}$ proves $t = t$.

Introduce.

$$\frac{\Gamma, B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}, \qquad \eta = \begin{cases} z, & B \text{ has equation root and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{I}$$

The witness t is canonical and z is fresh for the retained context, the witness, and all passive matrix parameters. Literal syntax, not provable equality, determines the mode.

Open with a synthesized name. For a major $D_{x,y}B(x,y)$ choose a fresh parameter b and put

$$B_b(y) = N(B(b,y)), \qquad \tau_B(b) = \begin{cases} r, & B_b(0) \equiv H \text{ and } B_b(1) \equiv (b = r), \\ b, & \text{otherwise.} \end{cases} \tag{P}$$

Define the substituted clause

$$\mathcal{C}^*_B(b) = [[N(B(\tau_B(b), 0))]; y.N(B(\tau_B(b), y))].$$

Then the rule is

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, \mathcal{C}^*_B(b) \vdash A}{\Gamma \vdash A}, \qquad b \notin \mathrm{FV}(\Gamma, A). \tag{O*}$$

The parameter b is also fresh for the passive parameters of B. The synthesized r may contain b; the branch still lives in the scope with b available, and its conclusion must be in the outer scope. The primitive rule has exactly two proof premises. There is no additional computation, equality, hypothesis, induction, cut, or substitution rule. Computation is part of canonical syntax, and reflexivity is the explicit fixed axiom clause.

Lemma 2.1 (Direct ordinary soundness)**.** *O^* is sound in intuitionistic first-order equality logic over the scalar computation equations.*

Proof. Open the existential major at b, obtaining $\mathcal{C}_B(b) = [[B_b(0)]; y.B_b(y)]$. If P selects its first branch, the zero instance is reflexive and clause use at 1 gives $b = \tau_B(b)$. Otherwise reflexivity gives the same equality because $\tau_B(b) = b$. Ordinary equality transport converts the whole clause reading $\mathcal{C}_B(b)^\circ$ to $\mathcal{C}^*_B(b)^\circ$. Apply the supplied branch derivation to this converted assumption. The conclusion contains no b, so ordinary existential elimination closes. Equality transport and simultaneous substitution are valid also for an equality $b = r(b)$. □

2.2 The preliminary rules that do not need general substitution

Weakening by clause occurrences, injective fresh renaming, and cut of an ordinary formula assumption are admissible by direct induction on derivations. For renaming, the injectivity and freshness preserve and reflect the literal patterns in I and P; normalization is equivariant. For ordinary-assumption cut, replace only uses of the selected unconditional constant clause, retain all other clause occurrences, and freshen local parameters before insertion. These arguments do not substitute arbitrary numerical terms into a derivation.

Lemma 2.2 (Two immediately ordinary forms of opening)**.** *O^* is exactly ordinary paired elimination in each of the following cases:*

1. *the witness coordinate x is unused in the matrix $B(x,y)$;*
2. *the matrix B has compound, rather than equation, root.*

Proof. In the first case $B_b(1)$ contains no b, so its left-hand term cannot literally be the fresh parameter b. In the second case $B_b(0)$ still has compound root, because term normalization does not erase formula binders, and therefore cannot be H. In either case P returns b. □

For unused w write $q_y A(y) = D_{w,y}A(y)$.

Lemma 2.3 (Closures and numerical cases)**.** *Before using general ordinary elimination or equality transport, the following rules are available:*

$$\frac{\Gamma, A(0) \vdash A(Sz)}{\Gamma \vdash q_y A(y)}, \qquad \frac{\Gamma \vdash q_y A(y) \qquad \Gamma \vdash A(0)}{\Gamma \vdash A(t)}.$$

Consequently $\Gamma \vdash A(0)$ *and* $\Gamma \vdash A(Sz)$*, with* z *fresh, imply* $\Gamma \vdash A(t)$ *for every well-scoped* t*.*

Proof. Introduce the unused witness of q at 1, forcing I's case mode. For specialization, use the unused-witness case of Lemma 2.2, which opens the clause $[[A(0)]; y.A(y)]$; U at t with the given base proof gives the desired outer conclusion. The numerical-case rule follows by weakening the successor proof with the unused assumption $A(0)$ and composing the two constructions. □

Lemma 2.4 (Unrestricted case introduction)**.** *For every canonical matrix* B *and every canonical witness* t*, the rule*

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y} B(x,y)}$$

is derivable, with the same freshness condition on z *as in I. Its derivation uses only U, I, and the two ordinary opening forms of Lemma 2.2.*

Proof. The primitive I already suffices unless B has equation root and $t = 0$. Write the remaining matrix as $E(x, y)$. Put

$$C(v, a, c) = N(I(v, a; u.c)), \quad u \notin \mathrm{FV}(c),$$

and for an equation $E = (s = t)$ define

$$G_v E = (C(v, 0, s) = C(v, 0, t)), \qquad L(E) = q_v G_v E.$$

The literal endpoints are $G_0 E = H$ and $G_{Sw} E = E$. Lemma 2.3 gives $E \vdash L(E)$ by closure introduction and $L(E) \vdash E$ by specialization at 1 and reflexivity of H. Only unused-witness opening is involved.

Let $A(x, y) = L(E(x, y))$. Under $A(0, 0)$, unwrap to $E(0, 0)$, insert the supplied premise by ordinary-assumption cut, and wrap $E(0, Sz)$ as $A(0, Sz)$. Since A is compound, primitive I at zero uses cases and gives $D_{x,y} A(x, y)$. Open this major at fresh c using the compound-matrix case of Lemma 2.2. To introduce $D_{x,y} E(x, y)$ at witness c, assume $E(c, 0)$, wrap it to $A(c, 0)$, use the opened clause at Sz, and unwrap $A(c, Sz)$ to $E(c, Sz)$. The fresh witness c is not literal zero, so I uses cases. The resulting $D_{x,y} E(x, y)$ is independent of c and can be exported. No general substitution theorem and no unrestricted opening rule was used. □

2.3 The substitution theorem and its noncircular proof

A simultaneous numerical substitution may send a parameter to a term containing that same parameter. Images are inserted once. Only the finitely many parameters appearing in a given derivation matter. As usual, bound variables and eigenparameters are freshened away from the images before the corresponding substitution is carried out.

Theorem 2.5 (Admissibility of arbitrary numerical substitution)**.** *If* $\Gamma \vdash A$ *and* σ *is any well-scoped simultaneous numerical substitution, then*

$$N(\Gamma\sigma) \vdash N(A\sigma).$$

This is an effective transformation of finite derivations. It is not a claim that substitution preserves every primitive rule instance.

Proof. Use strong induction on the original derivation, with the induction assertion quantified over *all* simultaneous substitutions. This quantification is essential: in one opening case the branch receives a different combined substitution from the major. The induction hypothesis is applied to original proper subderivations, never to an already translated derivation of potentially larger size.

For U, substitute its selected clause occurrence and its zero or one premises, then use normalization–substitution coherence. The distinguished axiom $\mathcal{R}$ is unchanged because it has no free numerical parameter. For I, first freshen z away from all images of σ. A progress instance stays progressive: its equation root and literal-zero witness are preserved. A case instance stays a case instance unless an equation matrix has a previously nonzero witness whose substituted normal form is zero. In that exceptional case, the induction hypothesis gives exactly the premise of the derived unrestricted case rule in Lemma 2.4; apply that rule. That lemma was proved before the present substitution theorem.

It remains to treat O^*. Freshen its parameter b away from the passive data and from the finitely many substitution images. Write $B' = N(B\sigma)$ and $\tau' = \tau_{B'}(b)$. Translate the major by the induction hypothesis. There are two cases according to the *old* probe.

If the old probe selected r, its two literal identities were $B_b(0) = H$ and $B_b(1) = (b = r)$. Applying σ while leaving b fixed preserves these identities after normalization: H is closed and b is untouched. Formula roots are preserved. Therefore the new probe is also successful and $\tau' = N(r\sigma)$ literally. Apply the induction hypothesis to the original branch using σ with b fixed. Its substituted clause is, by coherence,

$$[[B'(\tau', 0)]; y.B'(\tau', y)],$$

which is precisely the clause needed by the new O^* instance.

If the old probe used its fallback, its branch clause was the ordinary $[[B(b, 0)]; y.B(b, y)]$. The new probe may remain false or may become true. In *both* cases apply the induction hypothesis once to the original branch with the combined simultaneous substitution that agrees with σ on the outer parameters and sends b to τ'. Its image τ' is inserted once, including when it contains b. The resulting clause is exactly $[[B'(\tau', 0)]; y.B'(\tau', y)]$. The outer context and branch conclusion become $N(\Gamma\sigma)$ and $N(A\sigma)$ because neither originally contained b, and the freshening ensured that no outer image introduces b. This supplies the new branch whether or not the guard was newly created. Close the new O^* instance.

This exhausts the rules. The only proof transformation invoked outside the induction hypotheses was the already established case repair. □

Corollary 2.6 (Unrestricted ordinary paired elimination)**.** *For arbitrary canonical B, the rule*

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad \Gamma, [[B(b, 0)]; y.B(b, y)] \vdash A}{\Gamma \vdash A}, \qquad b \notin \mathrm{FV}(\Gamma, A),$$

is admissible, where b is also fresh for the passive parameters of B.

Proof. Apply Theorem 2.5 to the supplied branch, with the simultaneous substitution $b \mapsto \tau_B(b)$ and all outer parameters fixed. Since Γ, A are b-free, the result is exactly the branch of O^* with the same outer conclusion A. The major is unchanged. This proves general ordinary elimination only *after* the substitution theorem, avoiding circularity. □

2.4 Recovering every equality rule

With C as above define

$$P(y) = N(I(y, 0; u.C(u, 1, 0))).$$

Computation gives $P(1) = 1$ and $P(2) = 0$ literally; no parity induction is needed.

Theorem 2.7 (Uncrossed transport)**.** *For every fixed numerical template $h(w)$,*

$$\Gamma \vdash s = t, \quad \Gamma \vdash N(h(s)) = r \quad \Longrightarrow \quad \Gamma \vdash N(h(t)) = r.$$

Proof. Use the canonical equation matrix

$$B(x, y) = \big(C(y, 0, C(P(y), N(h(x)), x)) = C(y, 0, C(P(y), r, t))\big).$$

Its zero, first, and second rows are respectively

$$H, \qquad x = t, \qquad N(h(x)) = r.$$

At witness s, numerical cases for $C(v, N(h(s)), s) = C(v, r, t)$, with the two supplied premises as its zero and successor cases, give the successor row at $v = P(Sz)$. This ignores I's local assumption, so I introduces $D_{x,y}B(x, y)$ in either mode. Open that major. Its probe returns t, so its branch clause is

$$[[H]; y.B(t, y)].$$

Reflexivity proves H, and U at 2 proves $N(h(t)) = r$ directly. That equation is in the outer scope, so O* closes. The numerical-case construction was derived before equality, and the transport construction does not require a general substitution of the supplied equality proofs. □

Symmetry follows by taking $h(w) = w$ and the reflexive minor $s = s$; crossed transport follows by symmetry of the output. Transitivity and term congruence follow in the usual finite equational constructions. Replacing both sides of an equation uses two one-term transports and symmetry.

Full formula transport follows by external induction on formula depth. For a compound template $D_{x,y}B(v, x, y)$, open the source formula by Corollary 2.6 at a fresh c, and introduce the target formula at witness c, which forces case mode. Under its zero assumption, transport backward in the smaller matrix, specialize the source clause at Sz, and transport forward in the smaller matrix. Close introduction and ordinary elimination. This uses no object-theory induction on quantified formulas.

2.5 Identity-language equivalence and both interpretation composites

Let $\mathcal{T}$ be the inherited fully normalized clause calculus with the same syntax, Ax, U and I, together with ordinary paired elimination and crossed one-term transport. Let $\mathcal{O}$ be the new three-rule calculus U, I, O*.

Theorem 2.8 (Exact equivalence on arbitrary clause sequents)**.** *For every canonical finite clause context Γ containing Ax and every canonical formula A,*

$$\Gamma \vdash_{\mathcal{O}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{T}} A.$$

Both translations are effective and leave the full end sequent literally unchanged.

Proof. From $\mathcal{T}$ to $\mathcal{O}$, translate proof trees recursively, replacing ordinary elimination by Corollary 2.6 and crossed transport by Theorem 2.7 and derived symmetry. U and I are shared. The preliminary admissibility theorems were proved internally in $\mathcal{O}$ and do not rely on this translation.

Conversely, $\mathcal{T}$ has full formula transport by the same formula-depth argument. Translate an O^* major and open it ordinarily at b, retaining its clause $\mathcal{C}_B(b)$. Derive $b = \tau_B(b)$ by reflexivity and U at 1 when the probe succeeds, or by reflexivity alone otherwise. In this opened context derive $q_y B(\tau_B(b), y)$ as follows. Under its zero assumption $B(\tau_B(b), 0)$, transport backward along the derived equality to $B(b, 0)$; use the old clause at Sz; transport forward to $B(\tau_B(b), Sz)$; and introduce the unused q witness at 1. Ordinary elimination of that q opens exactly $\mathcal{C}_B^*(b)$. Insert the translated O^* branch, weakened by the still-retained old clause, and close this unused-witness elimination. The branch conclusion is free of b, so the outer ordinary elimination also closes. All operations are on finite proof trees, with fresh parameters chosen before insertion. □

Corollary 2.9 (Full iPRA and the identity numerical composites)**.** *The new calculus is equiconsistent and bi-interpretable with full first-order intuitionistic primitive recursive arithmetic. Both interpretations use the whole numerical domains and actual equality; both numerical composite isomorphisms have graph $x = y$.*

Proof. Compose Theorem 2.8 with the inherited canonical clause/native-scalar comparison and the inherited scalar/iPRA bi-interpretation. The new comparison is identity on every numerical term and formula, so it changes neither numerical map nor the formula translations. Each new composite therefore consists of identity legs and an inherited composite already provably equivalent to the original formula, with numerical graph $x = y$. The designated contradiction $0 = S0$ is unchanged, giving equiconsistency as a special case. □

The new universal metatheorems are Theorems 2.5 and 2.8. Their dependency order is: structural renaming and ordinary-assumption cut; the two guard-free openings; closure rules; case repair; strong-induction substitution; unrestricted ordinary opening; equality; exact comparison. The iPRA arithmetic compiler and normalization foundation remain explicitly inherited results. No newly proof-assistant-checked status or primitive-recursive bound on full normalization is asserted.

3 The normalization layer is part of the theory

Moving computation into syntax removes two displayed computation axioms, but it does not remove their implementation. The checker includes full normalization for exactly

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u],$$

closed under every term context, including iterator bodies. There is no reduction of formula constructors. The following argument records the inherited foundation [3, 2] so that the new probe does not depend on an unstated computation oracle.

Lemma 3.1 (Closure under iteration)**.** *Suppose n and a are strongly normalizing, and $h[v/u]$ is strongly normalizing for every strongly normalizing v. Then $I(n, a; u.h)$ is strongly normalizing.*

Proof. For a strongly normalizing term s, let $\nu(s)$ be the maximum length of a reduction from it. It exists because the reduction tree is finite branching and has no infinite path. In particular h is strongly normalizing by taking $v = u$. Induct lexicographically on

$$(\nu(n),\ |n|,\ \nu(a) + \nu(h)).$$

A reduction inside n decreases the first component. A reduction inside a or h decreases the last; the universal hypothesis on h survives a reduction in h, because capture-free substitution

preserves reduction. The zero root reduct is a. At a successor root $n = Sp$, the predecessor p has the same reduction height and smaller size. Thus $I(p, a; u.h)$ is strongly normalizing by induction, and its substitution into h is strongly normalizing by the hypothesis on h. Every immediate reduct is strongly normalizing, which proves the claim. □

Theorem 3.2 (Unique normal forms and substitution coherence)**.** *Every finite raw term has a unique normal form up to bound-variable renaming. With de Bruijn indices the normal form is unique literally. For every finite simultaneous substitution σ,*

$$N(t\sigma) = N\big(N(t)[x \mapsto N(\sigma x)]\big). \qquad \text{(Coherence)}$$

The same identity holds for formulas and clauses, and normalized substitutions compose in the corresponding capture-avoiding way.

Proof. Structural induction on t proves the stronger statement that every substitution of strongly normalizing terms into t yields a strongly normalizing term. The iterator case uses the preceding lemma; the induction hypothesis on its body applies with every additional strongly normalizing assignment to the state variable. Identity substitution gives strong normalization of all raw terms.

For local confluence, disjoint contractions commute, and the zero and successor root patterns do not overlap. If a root contraction copies a subterm containing the other contraction, perform that contraction in each of its finitely many residual copies. If it erases the subterm, no residual contraction is required. Capture-free substitution supplies the common reduct in the iterator-body case. Induction on reduction height now proves confluence. A substituted reduction of t and the reductions of the finitely many substituted images give convertible terms on the two sides of Coherence; their unique normal forms agree. Formula and clause constructors are preserved, so the same proof extends componentwise to them. □

Connection with raw syntax. In the raw scalar calculus each reduction is a provable equation by a computation axiom and congruence. Therefore every fixed term is provably equal to its normal form, and every formula is provably equivalent to its normalized formula. In the opposite direction the normal forms of both computation axioms are reflexive equations. Transport and clause use commute with normalization by Coherence. The only subtle introduction case is a nonzero-looking raw witness that becomes literal zero; the compound-envelope case repair supplied above handles that instance. These observations give the canonical/raw comparison used in Corollary 2.9. The raw formulas and their normal forms represent the same ordinary first-order statements up to provable equivalence.

No uniform numerical evaluator is postulated. For each fixed finite iterator description, iPRA can interpret its operation by primitive recursion. That does not give a primitive-recursive function uniformly evaluating all primitive-recursive descriptions. In fact such a uniform evaluator would contradict the familiar diagonal construction $n \mapsto U(n, n) + 1$. The normalization theorem above is a metatheorem about all finite terms; it is not an extra totality axiom inside iPRA. The complete checker includes the normalization code, and the fuelled checker supplies a bounded certificate relation without claiming a uniform primitive-recursive normalization bound.

Why canonical syntax is checked, not merely evaluated. Input terms are required to be normal forms. The syntax traversal checks their constructor shapes and variable scopes,

computes their normal forms, and requires the result to be identical to the input. Rule-created substitution instances may expose new redexes; those are normalized before use. In particular the probe uses normalized zero and unit instances, not a textual search through an unnormalized matrix. It can therefore discover a name that becomes visible only after computation.

4 The full intuitionistic theory and both composites

Exact equivalence of clause sequents is stronger than equality of closed equational theorems. Here we spell out how it transfers the complete iPRA interpretation, including its logical operations and both round trips. This section summarizes inherited results [1, 5, 6]; their detailed written arithmetic proofs are included in `reference`.

4.1 Ordinary realization

Let T_I be intuitionistic first-order logic with actual equality, genuine falsity $\bot$, and the staged scalar-iterator language. Its nonlogical axioms and schemas are the two iterator computations,

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot,$$

and induction for every literal equation $E(y, \vec{p})$:

$$(E(0, \vec{p}) \wedge \forall y(E(y, \vec{p}) \to E(Sy, \vec{p}))) \to \forall y E(y, \vec{p}).$$

Equivalently this is the usual contextual induction rule, with the induction variable fresh for retained assumptions. A staged iterator symbol $i_h(n, a, \vec{p})$ is introduced for each fixed earlier-stage description $h(u, \vec{p})$; no object-language variable ranges over arbitrary function descriptions. The numerical domain is the whole domain and interpreted equality is equality itself.

Use has its ordinary conditional-universal reading. Introduce is sound by numerical coverage in case mode and equation induction in progress mode. Opening is sound by Lemma 2.1. The explicit falsity bridge must not be omitted from T_I: positive arithmetic equations alone admit a one-element structure. In the native calculus the designated contradiction is the equation $0 = S0$.

4.2 All logical operations as macros

Write $C(v, a, b) = N(I(v, a; u.b))$ with u unused, and let $q_y A(y) = D_{w,y} A(y)$ for unused w. A gate on formulas is defined by external recursion on their finite syntax:

$$G_v(s = t) = (C(v, 0, s) = C(v, 0, t)),$$
$$G_v(D_{x,y} B) = D_{x,y} G_v B.$$

All bound variables are renamed away from v. Its endpoints satisfy $\vdash G_0 A$ and $G_{Sz} A \dashv\vdash A$; also $A \dashv\vdash q_y G_y A$ when y is fresh. These are derivability statements, including for nested binders.

For equations define the selector

$$M_v(s = t, u = w) = (C(v, s, u) = C(v, t, w)).$$

For compound formulas align fresh paired binders and define

$$\begin{aligned} M_v(D_{x,y}A, D_{x,y}B) &= D_{x,y}M_v(A, B), \\ M_v(E, D_{x,y}B) &= D_{x,y}M_v(G_yE, B), \\ M_v(D_{x,y}A, E) &= D_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Each recursive call lowers the maximum binder depth. Gate padding and binder congruence give $M_0(A, B) \dashv\vdash A$ and $M_{Sz}(A, B) \dashv\vdash B$. With $P(y) = N(I(y, 0; u.C(u, 1, 0)))$, the following complete inventory is available:

$$\begin{aligned} \bot_* &= (0 = S0), \\ A \Rightarrow_* B &= q_y M_y(A, B), \\ A \wedge_* B &= q_y G_y M_{P(y)}(A, B), \\ \exists_* x\, A(x) &= D_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* v\, M_v(A, B), \\ \forall_* x\, A(x) &= A(0) \wedge_* q_y G_y A(y). \end{aligned} \qquad \text{(Logical macros)}$$

Freshness is understood in every line. The last line intentionally supplies the zero instance separately: the gated closure supplies all successor instances.

Implication introduction and elimination use selector endpoints and the closure rules. Existential introduction and elimination use gate endpoints. Conjunction is projected at 1 and 2, using $P(1) = 1$, $P(2) = 0$; introduction uses numerical cases on its selector. Disjunction uses control witnesses 0 and 1, and its elimination uses numerical cases. For universals, numerical coverage combines the zero conjunct with the successor instances in the closure. Under $0 = S0$, equality transport identifies $C(0, s, t)$ and $C(S0, s, t)$, proving every numerical equation; external induction on formula depth and case introduction then prove every formula. Thus all usual intuitionistic first-order inference rules, including explosion, are recovered.

These macros do not add unrestricted induction for quantified formulas. Primitive progress remains restricted to equations. Equation induction itself is obtained by introducing a closure at an unused zero witness, opening it, and supplying its base. The explicit repaired case rule handles the case introduction instances needed above.

4.3 The numerical arithmetic bridge

The inherited scalar proof has a necessary dependency order. First it constructs a scalar predecessor and proves $\mathrm{pred}(0) = 0$ and $\mathrm{pred}(Sn) = n$ using equation induction and coverage. It then constructs truncated subtraction and a characteristic term for actual equality on the entire domain. Compiling a fixed quantifier-free formula into its characteristic equation gives quantifier-free induction. Only after this stage does it construct pairing and compile arbitrary primitive recursion. These steps are proved internally, rather than checked only on standard numerals.

For concreteness, once predecessor, subtraction, and quantifier-free induction are available, a bounded square-root construction proves pairing and projections π, L, H satisfying

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

One choice is $\pi(a, b) = (a + b)^2 + a$; bounded search for the integer square root supplies its projections. Surjectivity of π is unnecessary: it stores a recursion state and does not replace the interpretation domain by a coding range.

Given a fixed primitive recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

and already translated g^*, h^*, set

$$\begin{aligned} W(u,\vec{p}) &= \pi(SL(u), h^*(L(u), H(u),\vec{p})),\\ V(n,\vec{p}) &= I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})),\\ f^*(n,\vec{p}) &= H(V(n,\vec{p})). \end{aligned}$$

Equation induction proves $L(V(n,\vec{p})) = n$ and the defining equations for f^*. Initial functions and composition translate directly. In the reverse direction, every fixed scalar iterator is an ordinary primitive recursion whose step ignores the recursion counter.

4.4 Why this is bi-interpretability

For the numerical composites, internal equation induction proves uniqueness of each fixed iterator or primitive recursion satisfying its defining equations. External induction over a finite function description therefore identifies its double translation with the original operation at every internal argument. The identity graph $x = y$ is consequently a definable total bijection preserving every operation in both composite interpretations. It is not a retract, quotient, or restriction to standard numerals.

For the logical composites, the essential native identity is

$$D_{x,y}B(x,y) \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y B(x,y)\big).$$

Forward, ordinary opening supplies the conditional-universal clause; introduce the three macros and close the opening. Backward, eliminate the existential macro, introduce D at its witness using the repaired case rule, and obtain the required successor instance from implication and universal elimination. The endpoint proofs above give the round trips for all ordinary connectives and quantifiers. External induction on formulas finishes both composites. Optional clauses correspond to their unconditional or conditional universal readings.

These are exactly the inherited bridges recorded by `biInterp` and the four faithfulness and formula round-trip theorems in `NativeBiInterpretation.lean` [5]. The new theorem preserves every end sequent of the canonical clause presentation, so the interpretations and the identity numerical graphs transfer unchanged. In particular the new calculus derives $0 = S0$ if and only if iPRA derives $\bot$. This establishes equiconsistency; it does not assert that either theory proves its own consistency.

5 Complete implementation and measured simplification

5.1 Input contract and trusted code

The primary program reads a JSON object with exactly two fields, `goal` and `proof`. The goal is a closed canonical formula. The checker prints `ACCEPT` only after inferring the proof's conclusion, validating its syntax and scope, and comparing it with the independently validated goal. An exception or nonzero process exit rejects. No Python `assert` is used in the kernel, so optimized execution does not disable a logical check.

Zero is integer `0`; negative integers are free parameter levels; positive integers are bound indices, with index one innermost. Terms `[0,t]` and `[n,a,h]` encode successor and iteration,

with one bound index added only inside `h`. A two-element formula `[s,t]` is an equation; `[B]` binds two indices in its matrix, with the counter at index one and the witness at index two. The sorted syntax traversal distinguishes a successor from an equation. Booleans, floats, out-of-scope indices, and noncanonical terms are rejected.

The instruction formats are

`["u",j,t,p]`	use clause j at t,
`["i",B,t,p]`	introduce $[B]$ at witness t,
`["e",p,q]`	infer a major from p and check branch q.

For unconditional clause use, the fourth payload is unused; the examples use JSON `null`. This is a declared certificate convention, not an unvalidated proof premise. Conditional use must infer exactly the retained antecedent. The context initially contains only `([], [1,1])`, the reflexivity clause. Every I or O* branch adds one clause and one parameter, so the context length determines the next fresh level without a separate numerical-scope argument.

Every successful recursive call validates its inferred formula in that call's outer scope. Thus opening cannot export its fresh parameter, including when the synthesized term contains it. Introduction returns the unchanged annotated matrix and validates that matrix at the end; malformed syntax temporarily erased in a computed branch cannot survive this final check. The independent validation includes such erased malformation examples.

The retained-antecedent invariant. The implementation computes the original antecedent $u = N(B(b, 0))$ and substitutes the synthesized name into the body. It need not recompute the antecedent. If the probe succeeds, u is literally H, and substitution leaves it unchanged. If it fails, the substitution is the identity. Therefore the stored pair is exactly the clause stipulated by O* in both cases. This identity accounts for part of the matched opening-control comparison below.

5.2 Source measurements

The measurement convention follows the repository: remove a module's documentation string, parse the remaining source, normalize it with CPython 3.12.14 `ast.unparse`, and append one newline. Count UTF-8 bytes, significant Python tokens (excluding comments, indentation, newlines and end markers), and all nodes from `ast.walk`. The gzip column uses level 9, a zero timestamp, and these normalized bytes. Gzip is an informational metric, not an optimization objective. All trusted syntax, normalization, binding, scope, proof, and JSON handling code is included. No untrusted proof-construction code is required for acceptance.

Complete program	Bytes	Tokens	AST	Gzip 9
Inferred opening (selected)	1,990	683	771	819
Matched equality opening	1,998	676	765	812
Matched equality output	1,983	663	751	812
Incoming witness discharge	2,273	732	813	903
Incoming uniform binding	2,238	693	764	866
Incoming clause contexts	2,282	666	770	774
Inferred opening with fuel	2,203	728	826	885
Incoming witness with fuel	2,486	777	868	967

Table 1: Complete programs under one reproducible source-measurement convention.

Against the incoming explicit-equality discharge implementation, the selected program saves 283 normalized bytes (12.45%), 49 significant tokens (6.69%), and 42 AST nodes (5.17%);

gzip falls by 84 bytes. This comparison includes both the new primitive interface and the change to a forward checker that infers conclusions once. It must not be attributed entirely to the mathematical rule restriction.

The matched explicit-equality *opening* control uses the same syntax traversal, normalizer, context convention, and forward interface. Its opening accepts an equality proof $b = t$, specializes the clause at t, and checks the branch. The new checker saves eight bytes, while using seven more significant tokens, six more AST nodes, and seven more gzip bytes. The matched explicit-equality *output* control instead keeps the unspecialized clause and substitutes into the branch's result; it is seven normalized bytes shorter than the selected checker. These mixed results are recorded rather than concealed: removing a proof premise does not automatically shorten all implementations.

The selected checker is shorter in normalized bytes than the six inspected explicitly typed incoming checkers, but not in every coordinate. For example, `uniform_binding` has seven fewer AST nodes, and `clause_contexts` has fewer tokens and one fewer AST node. The repository's 516-byte compact codec and 788-byte validated-public-goal implementation remain substantially shorter [1]. Their representation and internal validation contracts differ, so they are not matched controls; their smaller sizes nonetheless preclude a global source-size record claim for this package.

5.3 The fuel-bounded certificate relation

The second standalone checker adds an integer `fuel` field. It decrements fuel on every syntax/normalization traversal entry and proof call, rejecting before the counter becomes negative. Its full size is included in the table. For each accepted unbounded certificate, its finite computation uses some finite amount of fuel; conversely every fuelled acceptance follows the same checking computation. The two idealized checkers therefore accept proofs of exactly the same formulas when sufficient resources are supplied.

With input size and fuel fixed, the number of recursive entries is bounded. Each entry performs finite operations on syntax whose growth has a primitive-recursive bound in those two inputs. The fuelled certificate relation is consequently primitive recursive. This does not provide a primitive-recursive bound on the fuel needed to normalize all descriptions, and does not claim that iPRA proves the uniform normalizer total. As with the baseline Python programs, implementation stack, memory, and parser limits can reject a large mathematically valid certificate; such rejection never licenses acceptance of an invalid one.

5.4 Executed evidence

The final primary source has SHA-256

```
79509b47cbbf20df923cc7b3e86952df33dcb3ce15b71f8e1d798212805a1902
```

The reproduction suite accepts 12 concrete closed certificates and rejects 38 adversarial objects. Positive examples include both directions of conditional symmetry, automatic transport at zero and nonzero witnesses, nested iterator binders, nested paired formulas, a branch that genuinely requires the specialized clause, and a synthesized term containing the fresh parameter. The suite performs 12 exact translations into the explicit-opening control, 24 exact-fuel/one-less checks, and 114 command-line checks with and without Python optimization. It also checks a case in which outer substitution creates the naming guard: blindly retaining the old branch fails, while the branch prescribed by the substitution theorem succeeds.

An independent validator uses named bound variables, a separately written sorted parser, and numerical evaluation by ordinary loops. It passes 6,292 checks with no failures, including 700 normalization comparisons, 1,398 numerical semantic comparisons, 500 binder-opening comparisons, 400 simultaneous substitutions, three sets of 350 probe/clause/invariant checks, 1,500 arbitrary-JSON grammar comparisons, and targeted scope and malformed-input examples. The source hash and complete category counts are retained in the evidence JSON.

These executions address concrete implementation risks: capture, accidental Python numeric coercion, malformed erased syntax, wrong equality orientation, a nonzero reflexive base, a compound matrix, self-containing substitutions, and eigenparameter escape. They are finite tests. The universal soundness, substitution, completeness, and bi-interpretation conclusions rest on the mathematical proofs and the attributed inherited bridge, not on the number of successful tests.

A Selected complete checker

The following is the complete measured program after CPython `ast.unparse` normalization. The archive also contains the source, the fuel-bounded variant, its matched control, and the reproduction script. The normalizer, all trusted syntax and scope checks, proof checking, and the JSON interface are included in the measurement.

```
import json

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or (size in (2, 1 if form else 3) and (form or size == 3 or a[0] == 0)))
    a = [walk(b, f, d + (2 if size == 1 else int(size == 3 and i == 2)), size == 1) for i, b in
     enumerate(a)]
    if size == 3:
        c, b, h = a
        if c == 0:
            return b
        if type(c) is list and len(c) == 2:
            return sub(h, walk([c[1], b, h], lambda i, d: i))
    return a

def syntax(a, n, form=False):
    need(walk(a, lambda i, d: need(1 - n <= i <= d) or i, form=form) == a)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: walk(t, lambda j, e: j + d if j > e else j) if i == d + 1 else i - (i >
     d + 1), depth)

def prove(p, g):
    n = len(g)
    b = -n
    match p:
        case ['u', j, t, q]:
            need(type(j) is int and 0 <= j < n)
            syntax(t, n)
            u, B = g[j]
            if u:
                need(prove(q, g) == u)
            A = sub(B, t)
        case ['e', q, r]:
```

```
            D, = prove(q, g)
            B = sub(D, b, 1)
            u, E = (sub(B, 0), sub(B, [0, 0]))
            t0 = E[1] if u == [0, 0] and E[0] == b else b
            B = walk(B, lambda i, d: t0 if i == b else i)
            A = prove(r, g + [(u, B)])
        case ['i', B0, t, r]:
            syntax(t, n)
            B = sub(B0, t, 1)
            u = sub(B, b if len(B0) == 2 and (not t) else 0)
            need(prove(r, g + [([], u)]) == sub(B, [0, b]))
            A = [B0]
        case _:
            need(False)
    syntax(A, n, True)
    return A

def check(data):
    need(set(data) == {'goal', 'proof'})
    syntax(data['goal'], 1, True)
    need(prove(data['proof'], [([], [1, 1])]) == data['goal'])
if __name__ == '__main__':
    check(json.load(open(0)))
    print('ACCEPT')
```

B Reproduction and source attribution

Run `python3 reproduce.py` from the package directory. It recomputes source metrics and runs the delivered verification. Run `python3 build_pdf.py` to rebuild this PDF using an installed TeX distribution; an ordinary `latexmk` command is also documented in `README.txt`. Reproducing the checker validation does not require a network connection or a proof assistant.

The `reference` directory contains attributed upstream mathematical fragments for the native choice calculus, ordinary interpretation, predecessor, equality reflection, and primitive-recursion compilation. It also contains the two terminal Lean bridge modules and baseline checker snapshots used for comparisons. These are reference inputs, not new theorems of this package. The Lean modules import the upstream library; the two snapshots alone are not a standalone Lean project. `PROVENANCE.json` records exact source locations and hashes. `CLAIMS.json` separates new theorems, inherited results, finite implementation evidence, and claims deliberately not made.

References

[1] ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, README at commit `17898ea7e6a1`. Pinned ConciseIPRA source tree.

[2] ProveIt research package, *iPRA with equality at witness discharge*, `docs/incoming/ipra_witness_discharge_2026-11-11.zip`, at the same commit. Inherited full-normalization clause interface, two-row equality package, and zero-witness case repair.

[3] ProveIt research package, `Research/full_normalization/report.tex`, at the same commit. Termination, confluence, and substitution coherence for scalar iterator normalization.

[4] ProveIt research packages, `Research/uniform_clauses`, `Research/raw_clausal`, and the incoming `ipra_uniform_binding_2026-11-11.zip`, at the same commit. Clause interfaces and complete checker comparisons.

[5] ProveIt contributors, `Lean/ConciseIPRA/BiInterpretation.lean` and `NativeBiInterpretation.lean`, at the same commit. Theorems `biInterp`, `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff`.

[6] ProveIt research package, `Research/certified_zero/reference`, at the same commit. Native logical macros, scalar predecessor, equality reflection, quantifier-free induction, and general primitive-recursion compilation; reproduced reference fragments are identified in the package provenance.